

Monotone Growth of Content-Addressed Rule Closures

Conservativity, history independence, and a parsimony trilemma for incrementally generated knowledge structures

Behnam Sour, MD · ORCID 0009-0002-3176-1956 Orthopaedic Surgery, Shafa Yahyaean Hospital, Tehran · Bone & Joint Reconstruction Research Center, Iran University of Medical Sciences

Working paper · v2.1 · not peer-reviewed. Formal framework with a three-rung verification ladder: a machine-checked reference implementation (`growth_check.py`; 15/15 ledger checks), a randomized property suite (`test_growth_properties.py`; 14/14 properties on ~840 adversarial instances with shrinking), and a self-contained Lean 4 formalization (`GrowthClosure.lean`; kernel-checked, sorry-free, no `Classical.choice`). Every numeric value, address, fingerprint, benchmark time, and corpus statistic in this document is produced by the accompanying scripts, not illustrated by hand. Relative to v1, this revision repositions Theorem 11 against the CALM theorem following a prior-art sweep (§5, §9), adds empirical scale measurements and the verification ladder (§8), and replaces informal attributions with an existence-verified bibliography. v2.1 completes the machine-checked perimeter: Theorem 4 and the H1/H2 discharge are now kernel-checked (§8.3).

Abstract

We study the *growth problem* for content-addressed rule closures: a knowledge structure is generated as the least fixpoint $K(S)$ of a finite family of typed generating rules over a seed set S of atoms, every part is identified by a recursive cryptographic hash of its derivation content, and the question is what happens when the seed grows. Under four standing hypotheses — finitary rules, payload-determined side conditions, stratified acyclic generation, and content addressing — we prove that growth is **conservative** ($S \subseteq S'$ implies $K(S)$ embeds in $K(S')$ at identical addresses, so growth is auditable as a Merkle address-set difference), **incremental** ($K(S \cup D) = K(K(S) \cup D)$, computable by frontier-local, semi-naive iteration with no deletion or rederivation), **history-independent** (the canonical fingerprint depends only on the union of ingested batches, verified over all 13 ordered set partitions of a 3-element delta), and **continuous** ($K(\bigcup_i S_i) = \bigcup_i K(S_i)$ for directed families, licensing streaming semantics). We then prove the qualitative dynamics: admissibility is stable but inertness is not, so seed growth is the unique, append-only repair channel for ill-formed structures (absorption); well-formedness itself is *not* monotone and must be re-checked by one breadth-first search per ingestion; and the first Betti number of the underlying graph is monotone non-decreasing along generation, with per-instance increment exactly $m - j$. Finally we isolate the cost of these guarantees: descriptor cohorts of size n generate $\sum_{m \geq m_0} \binom{n}{m}$ admissible bindings (exponential; 42 at $n = 6$), and we localize a known monotonicity boundary — the CALM theorem of Hellerstein and of Ameloot, Neven & Van den Bussche — in this rule format: no payload-determined side condition can store exactly the $\subseteq$ -maximal bindings at every seed (Theorem 11, with a kernel-checked decidable witness), giving a trilemma: conservativity, history independence, and extensional maximal parsimony are jointly unsatisfiable. Three resolutions are developed (subsumption-quotient views, versioned cohort objects, and two-phase closure-then-selection), all instances of one design principle: *keep identity and storage monotone and complete; spend parsimony in views*. The full closure algebra — including semi-naive frontier locality, with monotonicity and finitariness derived from the rule format rather than assumed — and the impossibility witness are machine-proved in a self-contained, constructive Lean 4 development; on a natural-language corpus the clique blow-up is real

(one 268-atom cohort alone implies $\sim 10^{80.7}$ admissible bindings), and a fixed 20-atom delta integrates into bases up to 8,000 atoms 6–10 $\times$ faster incrementally than by batch reclosure.

1. Introduction

A generated, content-addressed knowledge structure faces a constraint that hand-built structures do not: it must integrate new material **without rebuilding and without rewriting history**. If every part’s identity is a hash of its derivation content, then any update discipline that retracts or rewrites parts breaks every reference ever issued to them; and if the structure is the value of a fixpoint, any update discipline that recomputes from scratch wastes everything already derived. The growth problem is to characterize exactly which guarantees an incrementally fed closure can have, at what cost, and where the unavoidable trade-off sits.

This paper answers the question for a general class of systems: typed directed graphs generated as least fixpoints of finitary rule signatures with payload-determined side conditions, under recursive content addressing. The class includes, as instances, Datalog-style saturation with content-addressed Skolemization, Merkle-DAG knowledge stores fed by aggregation rules, and compression-gated (MDL-admissible) structure induction over growing corpora. The theorems do not depend on any particular type tower; a small four-level demonstration tower is used only to make every claim machine-checkable.

Contributions.

1. **Growth theorems** (§3). Conservativity with address-stable embedding (Thm. 2); the incremental-closure identity $K(S \cup D) = K(K(S) \cup D)$ (Thm. 3); frontier locality, i.e. correctness of semi-naïve delta evaluation with no deletion phase (Thm. 4); ingestion confluence — the fingerprint is a function of corpus content, not ingestion history (Thm. 5); and Scott-continuity in the seed, licensing streaming (Thm. 6).
2. **Qualitative dynamics** (§4). Stability of activation vs. instability of inertness; absorption as the unique append-only repair mechanism; non-monotonicity of well-formedness with a one-BFS re-check discipline (Prop. 8); monotone growth of the first Betti number with the exact per-instance law $\Delta\beta_1 = m - j$ (Prop. 9).
3. **A localized impossibility boundary** (§5). The descriptor-clique blow-up is quantified (Prop. 10); Theorem 11 proves that no payload-determined side condition realizes exactly the maximal-binding field at every seed — positioned, following the prior-art sweep, as a content-addressed, MDL-flavored instance of the CALM boundary (Hellerstein 2010; Ameloot, Neven & Van den Bussche 2013; Hellerstein & Alvaro 2020) rather than a new phenomenon. Corollary 12 states the resulting trilemma, whose conservativity vertex — address-stable embedding — has no CALM counterpart, with machine-checked witnesses including a real retracted address.
4. **Resolutions** (§6). Three architectures that each sacrifice a different vertex of the trilemma, with their guarantees stated precisely.
5. **A three-rung verification ladder and empirical scale** (§7–§8). A pure-stdlib reference implementation reproduces every quantitative claim (real SHA-256 fingerprints, confluence over all 13 ingestion histories, the retraction counterexample, the 42-block clique, $\beta_1 = 150$, an 11×11 threshold grid); a randomized property suite re-tests the universal statements on ~ 840 adversarial instances; a self-contained Lean 4 development kernel-checks the closure algebra constructively — Theorem 4 included, over a concrete payload-determined rule format with H1/H2 discharged as theorems — and the impossibility witness (by decide); and two measurements quantify delta-integration cost and the clique blow-up on a natural corpus.

Relation to prior frameworks. The setting axiomatizes, in neutral terms, the growth behaviour of any typed generative-closure system satisfying H1–H4 below. The components

are individually classical — least fixpoints, semi-naive evaluation, the chase, MDL, Merkle stores; §9 gives the lineage. The contribution claimed is the growth analysis itself: the four guarantees, the impossibility result that bounds them, and the verified implementation.

2. Formal setting

2.1 Atoms, types, parts

Definition 1 (atom). An *atom* is a pair $a = (p, h)$ where p is a payload string and $h = \text{SHA-256}(p)$ is its address. Each atom carries a *descriptor* $d(a)$, a designated payload field naming the shared feature class it instantiates. Atoms are the generators: no rule outputs type Atom.

Definition 2 (type poset). T is a finite graded poset with grade $\rho : T \rightarrow \mathbb{N}$, Atom the unique element of grade 0, and a terminal constant $\top$ above all types, reached only by governance edges. The *demonstration tower* used throughout for machine checks is

$$\text{Atom}(0) < \text{Block}(1) < \text{Section}(2) < \text{Root}(3); \quad \top \text{ constant.}$$

Definition 3 (Σ -graph). A Σ -graph is a typed directed graph $G = (V, E, \tau)$, $\tau : V \rightarrow T$, in which every non-atom vertex is the output of exactly one instance of a rule from a finite signature Σ , with edges from the instance’s premises to its output, plus governance edges into $\top$.

Definition 4 (rule). A rule $R \in \Sigma$ is a tuple $R = (\text{in}_R, \text{out}_R, \varphi_R, \pi_R)$: a finite list of input types, an output type, a side condition φ_R , and a payload constructor π_R . The demonstration signature:

rule	typing	side condition	kind
bind	$\text{Atom}^m \rightarrow \text{Block}$	$m \geq 3$, shared descriptor s , $\Delta(s, m) > 0$	ascent
lift	$\text{Block} \rightarrow \text{Section}$	—	ascent
frame	$\text{Section} \rightarrow \text{Root}$	—	ascent
close	$\text{Root} \rightarrow \top$	—	governance

Here $\Delta(s, m) = (m - 1)b(s) - o - mr$ is the two-part-code (MDL) saving of storing the shared descriptor once (at codebook overhead o , descriptor information $b(s)$ bits, reference cost r per use); a binding is *admissible* iff it strictly compresses, $\Delta > 0$. The default code parameters in the ledger are $b = 2$, $o = 2$, $r = 0$, giving MDL threshold $m_0 = \lfloor o/b \rfloor + 2 = 3$ (verified on an 11×11 grid, §7H).

2.2 Standing hypotheses

All results assume:

- **H1 (finitarity).** Every rule instance has finitely many premises.
- **H2 (payload determinism).** Every side condition φ_R — including the descriptor function, code lengths, and Δ — is a function of the premise payloads alone, fixed independently of the ambient graph.

- **H3 (stratified acyclic generation).** Ascent rules strictly increase grade; the generating fragment below any part is a finite DAG; $\top$ is a constant identifier outside the hash recursion, so governance edges create no content cycle.
- **H4 (content addressing).** Addresses extend recursively from atoms to all parts:

$$\text{addr}(v) = \begin{cases} \text{SHA-256}(p) & v = (p, h) \text{ an atom,} \\ \text{SHA-256}(\text{kind}(v) \parallel \text{rule}(v) \parallel \text{descr}(v) \parallel \text{sort}\{\text{addr}(c) : c \in \text{in}(v)\}) & \text{otherwise,} \end{cases}$$

with $\text{addr}(\top) = \text{SHA-256}(\text{"TOP"})$ fixed. Parts are identified by address: duplicate derivation content collapses to one node.

H2 is the load-bearing hypothesis. In a fully context-sensitive MDL code the cost of a part depends on the global codebook, so admissibility can flip as the field grows; restricting to payload-determined codes is precisely what purchases monotonicity (Lemma A below) — and §5 proves that this purchase is unavoidable.

2.3 Closure as a least fixpoint

Fix a countable atom universe; since Σ is finite and rules are finitary, the universe $\mathcal{U}$ of derivable records is countable, and $(\mathcal{P}(\mathcal{U}), \subseteq)$ is a complete lattice.

Definition 5 (one-step expansion). $\Phi(X)$ is the set of outputs of all rule instances whose premises lie in X and whose side conditions hold. The *inflationary* form is $\hat{\Phi}(X) = X \cup \Phi(X)$.

Lemma A (persistence). Under H2, if a side condition holds for premises lying in X and $X \subseteq Y$, it holds for the same premises in Y . *Proof.* The condition is a predicate on premise payloads; payloads do not change when the ambient set grows, and the premises persist into Y . ■

Lemma B (monotonicity, continuity). Φ and $\hat{\Phi}$ are monotone; under H1 they are Scott-continuous: $\hat{\Phi}(\bigcup_i X_i) = \bigcup_i \hat{\Phi}(X_i)$ for directed families. *Proof.* Monotonicity from Lemma A. Continuity: any output of $\hat{\Phi}(\bigcup_i X_i)$ has finitely many premises, all in a single X_j by directedness. ■

Lemma C (existence, constructivity). For any seed S the closure $K(S) = \mu X. (S \cup \Phi(X))$ exists and equals $\bigcup_{k \geq 0} \hat{\Phi}^k(S)$. *Proof.* Knaster-Tarski gives the least fixpoint of the monotone $X \mapsto S \cup \Phi(X)$ on the complete lattice; Kleene's theorem with Lemma B gives the constructive ω -union. ■

2.4 Canonical form and fingerprint

Definition 6 (canonical serialization). $\sigma(G)$ lists every part as the record $\langle \text{addr}, \text{kind}, \text{rule}, \text{descr}, \text{sorted} \rangle$ sorted by (grade, kind, address); governance edges are appended in sorted order. The *fingerprint* is $\text{fp}(G) = \text{SHA-256}(\sigma(G))$.

Lemma D (order independence). $\text{addr}(v)$ depends only on the derivation content of v , not on when v was derived; hence the record set of $K(S)$, and therefore σ and fp , are independent of the order in which the fixpoint iteration fires rules. ■

The fingerprint of the worked five-atom field of §7A is

1066bddea3ff771058f4af6afc828efbeeb29f2968cf3c68cce02110cf64d9662

with all part addresses pairwise distinct (injectivity checked, including $\top$).

3. Growth theorems

Throughout, $S \subseteq S'$ are seeds of atoms and D is a finite *delta* of new atoms.

Theorem 1 (closure operator). K is a closure operator on seeds and on Σ -graphs: extensive ($X \subseteq K(X)$), monotone ($X \subseteq Y \Rightarrow K(X) \subseteq K(Y)$), idempotent ($K(K(X)) = K(X)$). *Proof.* Extensivity by definition of the fixpoint. Monotonicity: larger seeds give larger values of the monotone map at every Kleene stage (Lemma B). Idempotence: $K(X)$ is Σ -closed, $\Phi(K(X)) \subseteq K(X)$, so re-closing adds nothing. ■

Theorem 2 (conservativity; growth is a Merkle diff). If $S \subseteq S'$ then: (i) $K(S) \subseteq K(S')$ — no part is ever retracted; (ii) every part of $K(S)$ occurs in $K(S')$ at the *identical address*, so $K(S)$ is an address-stable sub-DAG of $K(S')$; (iii) the record set of $\sigma(K(S))$ is a subset of that of $\sigma(K(S'))$, and the growth is exactly the address-set difference $\text{addr}(K(S')) \setminus \text{addr}(K(S))$. *Proof.* (i) is Theorem 1's monotonicity, which rests on Lemma A: under H2 a new atom can never invalidate an old instance's side condition, and closure only adds. (ii) Addresses (H4) are functions of derivation content alone, which is unchanged by ambient growth. (iii) immediate from (i)–(ii) and Definition 6. ■

Consequences. Old references never dangle; checkpoint-to-checkpoint growth is auditable by comparing two sorted address sets, with no replay; and Theorem 5 below makes the checkpoints themselves canonical. (Descent is also stable: since no rule outputs Atom, the atoms of $K(S')$ are exactly S' , so reading the seed back remains exact under growth.)

Theorem 3 (incremental closure). $K(S \cup D) = K(K(S) \cup D)$. *Proof.* ($\supseteq$) $S \cup D \subseteq K(S) \cup D \subseteq K(S \cup D)$ by extensivity and monotonicity; applying the monotone idempotent K : $K(K(S) \cup D) \subseteq K(K(S \cup D)) = K(S \cup D)$. ($\subseteq$) From $S \cup D \subseteq K(S) \cup D$, monotonicity gives $K(S \cup D) \subseteq K(K(S) \cup D)$. ■

One never recomputes from scratch: seed the delta into the already-closed field and re-close.

Theorem 4 (frontier locality; semi-naive correctness). Define the frontier iteration from a closed field: $F_0 = D \setminus K(S)$; F_{k+1} = the outputs, not already present, of rule instances with all premises in $K(S) \cup \bigcup_{j \leq k} F_j$ and **at least one premise in** $\bigcup_{j \leq k} F_j$. Then $K(S) \cup \bigcup_k F_k = K(S \cup D)$. Consequently the delta closure is computable by firing only instances that touch the frontier; there is no deletion phase and no rederivation. *Proof.* ($\subseteq$) every frontier output is derivable from $K(S) \cup D$, hence in $K(S \cup D)$ (Thm. 3). ($\supseteq$) Key step: if an instance has *all* premises in $K(S)$ and its side condition holds, its output is in $\Phi(K(S)) \subseteq K(S)$ already — closedness plus H2. So any part of $K(S \cup D) \setminus K(S)$ is produced by an instance with at least one premise outside $K(S)$. Induct on Kleene stage: a stage- $(k+1)$ part not in $K(S)$ has all premises at stage $\leq k$, each either in $K(S)$ or (by induction) in some F_j ; at least one premise is of the latter kind, so the instance fires in the frontier iteration. ■

Cost. With premise indexing by (descriptor, kind), work is proportional to the size of the delta closure times rule fan-in — the growth region only. The contrast with classical incremental view maintenance is structural: deletion-capable maintenance (e.g. DRed) must over-delete and rederive because facts can lose support; conservativity (Thm. 2) makes support loss impossible, so the entire deletion machinery vanishes.

Theorem 5 (ingestion confluence; history independence). Let D be a finite delta and $(D_1, \dots, D_k)$ any ordered partition of D into nonempty batches. Iterating incremental closure,

$$X_0 = K(S), \quad X_j = K(X_{j-1} \cup D_j),$$

yields $X_k = K(S \cup D)$, and hence the identical canonical form and fingerprint, for every partition and every order. Checkpoint fingerprints are therefore well-defined functions of

corpus *content*, not ingestion *history*. *Proof.* Induction on j with Theorem 3: $X_j = K(K(S \cup D_1 \cup \dots \cup D_{j-1}) \cup D_j) = K(S \cup D_1 \cup \dots \cup D_j)$. The fingerprint depends only on the record set of the final closure (Lemma D). ■

Machine check (§7B). For a 3-element delta there are exactly 13 ordered set partitions (the ordered Bell number $a(3) = 13$). All 13 histories — one batch, three sequential singletons in all orders, and every mixed batching — produce **one** fingerprint, equal to the single-batch fingerprint:

5c8942ed83c9b398554ea4898f04f4eebe28ac0796b6d099b9e67e49fe76d38a

Theorem 6 (continuity in the seed; streaming semantics). For any directed family of seeds $\{S_i\}$, $K(\bigcup_i S_i) = \bigcup_i K(S_i)$. *Proof.* By Lemma B each $\hat{\Phi}^k$ is Scott-continuous, so $\hat{\Phi}^k(\bigcup_i S_i) = \bigcup_i \hat{\Phi}^k(S_i)$ by induction on k ; then $K(\bigcup_i S_i) = \bigcup_k \bigcup_i \hat{\Phi}^k(S_i) = \bigcup_i K(S_i)$, the unions commuting. ■

The field of a growing corpus is the colimit of the stage fields: a streaming ingester computes, at every moment, a genuine sub-object of the eventual field, and nothing computed is ever wasted. (Theorems 2–6 are exactly the algebra of a grow-only replicated set over records: merge is union, which is associative, commutative, and idempotent — confluence is convergence.)

4. Qualitative dynamics of growth

Proposition 7 (activation is stable; inertness is not). If a rule instance is admissible over S , it is admissible over every $S' \supseteq S$ and its output persists (Lemma A, Thm. 2). The converse fails: an atom with no admissible instance over S (an *inert* or *orphan* atom) may acquire one over $S' \supset S$, namely when growth supplies the missing cohort members. Growth is therefore the **unique repair channel**: the graph is never edited; the seed is grown. Repair is append-only.

Definition (well-formedness). Let $\hat{G}$ be the undirected graph underlying G (content plus governance edges). G is *well-formed* iff $\hat{G}$ is connected to $\top$ and $\text{ecc}_{\hat{G}}(\top) \leq h$, where h is the ascent height of the tower ($h = 4$ for the demonstration tower: the extremal path is Atom-Block-Section-Root- $\top$). Decidable by one breadth-first search from $\top$.

Proposition 8 (well-formedness is not monotone). Growth neither preserves nor reflects well-formedness: (a) *repair by absorption* — an ill-formed field with an orphan atom becomes well-formed when the orphan’s descriptor cohort reaches the admissibility threshold and binds; (b) *breakage by novelty* — a well-formed field becomes ill-formed when a delta introduces an atom whose descriptor cohort is below threshold. Hence well-formedness is a **validation predicate, not a generation gate**: the closure always exists and all of §3 applies regardless; the discipline is one BFS re-check after every ingestion. *Proof.* (a) and (b) are realized by explicit fields; the ledger exhibits (a). ■

Machine check (§7C). Stage 1: cohort $\{c_1, c_2, c_3\}$ on D_1 plus one orphan on D_2 — **ill-formed** (1 orphan; $\text{ecc}(\top) = 4$ on the reached component). Stage 2: a 2-atom delta on D_2 completes the cohort; the orphan binds, lifts, frames, closes — **well-formed**, 0 orphans, $\text{ecc}(\top) = 4$. Stage-1 addresses are a subset of stage-2 addresses (conservativity), and the incremental fingerprint equals the batch fingerprint:

stage 1	2b2e50445f9e1bebf9a22de28b087b1424585deae6c3fa659b97e3d56b79866	ill-formed
stage 2	97f6082afaba7abafa97eb69a42f73989c4018e31d55f5ecbd6a7f77c80dd497	well-formed

Proposition 9 (the first Betti number is monotone; per-instance law). Let $\beta_1 = |E| - |V| + C$ on $\widehat{G}$ (C = number of connected components). Along any generation or growth trace: (i) adding a fresh atom: $\Delta\beta_1 = 0$; (ii) adding a composite with m premises spanning j distinct existing components: $\Delta V = 1$, $\Delta E = m$, $\Delta C = 1 - j$, hence $\Delta\beta_1 = m - j \geq 0$ (since $j \leq m$); (iii) adding a governance edge between existing nodes: $\Delta\beta_1 = 0$ if it merges two components, 1 if its endpoints were already connected. Therefore β_1 is non-decreasing under generation and under seed growth; since every connected graph deformation-retracts onto a wedge of β_1 circles, $\pi_1(\widehat{G})$ is free of rank β_1 throughout, and the homotopy type only ever *accumulates* circles. *Proof.* Direct computation of the Euler-type quantity in each case; $j \leq m$ because each premise lies in some component and there are m premises. ■

Machine check (§7G). On the $n = 6$ clique field: $V = 133$, $E = 282$, $C = 1$, $\beta_1 = 150$; a union-find replay of the entire generation reproduces $\beta_1 = 150$ and confirms $\Delta\beta_1 = m - j$ at every ascent instance. (Sanity: $V = 6 + 42 + 42 + 42 + 1$; $E = \sum_{m=3}^6 m \binom{6}{m} + 42 + 42 + 42 = 156 + 126$.)

5. The parsimony dilemma

The guarantees of §3 are purchased by H2, and H2 has a price. This section quantifies the price and proves it cannot be negotiated away within the rule format.

Proposition 10 (descriptor-clique blow-up). $\Delta(s, m) = (m - 1)b - o - mr$ is strictly increasing in m iff $b > r$. Hence once a descriptor pays at the threshold $m_0 = \lfloor o/b \rfloor + 2$ (for $r = 0$; verified on the 11×11 grid with the characterization $m_0 = 3 \iff o/b \in [1, 2)$), **every** subset of the cohort of size $\geq \max(3, m_0)$ is admissible. A cohort of n atoms therefore generates

$$N(n) = \sum_{m=\max(3, m_0)}^n \binom{n}{m} = 2^n - 1 - n - \binom{n}{2} \quad (m_0 \leq 3),$$

binding nodes — exponential in n — and each binding lifts and frames, so the redundancy propagates upward linearly per binding. *Machine check (§7F).* $N(3), N(4), N(5), N(6) = 1, 5, 16, 42$, exactly matching the closure (42 blocks $\rightarrow$ 42 sections $\rightarrow$ 42 roots at $n = 6$).

The obvious remedy is to bind **only** $\subseteq$ -**maximal** premise sets per descriptor. The next theorem shows this remedy is incompatible with the entire §3 package.

Theorem 11 (impossibility of payload-determined maximality). There is no side condition φ satisfying H2 such that, for every seed S , the closure under φ contains exactly the $\subseteq$ -maximal admissible bindings of each descriptor cohort in S . *Proof.* Take four atoms $a_1, \dots, a_4$ sharing one descriptor, with $m_0 = 3$. Let $t = (a_1, a_2, a_3)$. Over $S_3 = \{a_1, a_2, a_3\}$, t is the unique maximal binding, so exactness forces $\varphi(t) = \text{true}$. Over $S_4 = S_3 \cup \{a_4\}$, the premises of t have the same payloads, so by H2 $\varphi(t) = \text{true}$ still — yet t is no longer maximal in S_4 , so exactness forces t *out*. Contradiction. (If instead $\varphi(t) = \text{false}$, exactness already fails at S_3 .) Maximality of a premise set is irreducibly a property of the *ambient seed*, hence inexpressible payload-determined. ■

Corollary 12 (the growth trilemma). No system in this rule format simultaneously satisfies:

- **(C) conservativity** — stages embed address-stably, nothing is retracted (Thm. 2);

- **(H) history independence** — the stored state is a function of seed content only (Thm. 5);
- **(P) extensional maximal parsimony** — the stored state contains exactly the maximal bindings.

Indeed $(C) \wedge (P)$ is already contradictory: by Theorem 11’s witness, the maximal-binding states at $S_3 \subset S_4$ satisfy $M(S_3) \not\subseteq M(S_4)$ — the binding B_{123} is *retracted* — so any operator $\Phi_{\max}$ storing exactly $M(\cdot)$ is non-monotone, loses the Knaster-Tarski/Kleene route entirely, and dangles every reference into the retracted cone. The complete closure achieves $(C) \wedge (H)$; §6’s quotient view achieves $(H) \wedge (P)$.

Machine check (§7D). $\Phi_{\max}(S_3)$ contains B_{123} ; $\Phi_{\max}(S_4)$ does not. The retracted address is real:

retracted 362db034e4f12612265ac2e3... (B₁₂₃, descriptor D1)

and the Section lifted from B_{123} at stage 1 now references a part absent from the stage-2 state. On the identical growth, the complete closure contains B_{123} at **both** stages and is conservative.

Positioning. Theorem 11 should be read as a content-addressed, MDL-flavored instance of a known boundary rather than a new phenomenon. That extremal aggregation is non-monotone is classical: semantics for aggregates inside recursion require monotonicity with respect to a lattice order precisely because operators like “maximal set” break it (Ross & Sagiv 1992/1997). The general form of the boundary is the CALM theorem (conjectured by Hellerstein 2010; proved by Ameloot, Neven & Van den Bussche 2013; surveyed in Hellerstein & Alvaro 2020): a problem has a consistent, coordination-free distributed implementation iff it is monotone — where “consistency” is confluence over input orderings, our history independence, and where the coordination-free computations are captured by *oblivious* transducers, deciding without global knowledge, exactly as our payload-determined side conditions decide from premise payloads alone. Under this dictionary, Theorem 11 instantiates the only-if direction for maximal bindings, and Corollary 12 adds the element CALM does not speak of: identity. In a content-addressed store, the cost of non-monotonicity is not abstract “coordination” but the retraction of named, referenced parts. The contribution claimed here is accordingly the trilemma formulation with address-stable conservativity, the localization of the boundary at MDL codebook globality, and the mechanized witness — not the existence of the boundary itself.

6. Resolutions: where to spend the parsimony

All three architectures below keep the complete monotone closure for *identity* and differ in what they *store* or *present*. Each gives up exactly one vertex of the trilemma.

6.1 Subsumption-quotient view — keeps $(H) \wedge (P)$, spends (C) in the view only

Define on bindings: $B \sqsubseteq B'$ iff $\text{descr}(B) = \text{descr}(B')$ and $\text{premises}(B) \subseteq \text{premises}(B')$. Strict containment of finite sets is well-founded, so maximal elements exist. **Store** the complete field $K(S)$; **present** the quotient view $\pi(K(S))$: drop subsumed bindings and the parts derived *exclusively* from them (their derivation cones). Then:

- π is history-independent — it is a function of $K(S)$, which is (Thm. 5);
- π is maximally parsimonious at the binding level;
- π is **non-conservative across stages** — and harmlessly so: retraction lives only in the view; every address persists in storage, so references issued against the field never dangle.

Machine check (§7E). Cohort $\{g_1, g_2, g_3\}$ at stage 1: field 1 block, view 1 block (B_{123} visible). Add g_7 : field 5 blocks, view 1 block (B_{1237}); B_{123} has left the **view** but remains in the **field** at both stages.

6.2 Versioned cohort objects — keeps (P) and a chain-level (C), spends node-level (C)

Store one object per descriptor cohort (payload = the sorted member address list), with bindings materialized on demand; storage is linear in atoms + descriptors, eliminating the 2^n blow-up at rest. The cohort payload changes with the seed, so node-level conservativity is lost by construction; it is **recovered at the version level** by an append-only, hash-linked chain of cohort versions: each new version references its predecessor’s address, the chain embeds under growth, and only the *head pointer* moves. Fingerprints must then name the materialization policy, since the same chain admits multiple materialized fields. This resolution is the storage-layer analogue of the distribution-aware relaxations of CALM (Zinn, Green & Ludäscher 2012; Ameloot, Ketsman, Neven & Zinn 2015): granting the system knowledge beyond individual payloads — here, the cohort version chain — buys back behavior that obliviousness forbids.

6.3 Two-phase: monotone closure + global selection — keeps (C) $\wedge$ (H), states (P) as optimization

Phase 1: the complete monotone closure — identity, storage, and all of §3. Phase 2: global MDL selection of a minimum-cost covering subfamily of bindings per checkpoint. Selecting the cheapest set of bindings whose union covers the cohort is weighted set cover — NP-hard, with the classical greedy $\ln n$ -approximation — and is therefore run as a **non-monotone view recomputed per checkpoint**, never as a storage discipline. Conservativity of the candidate space (Thm. 2) means each re-optimization warm-starts from the previous solution plus only the frontier’s new candidates.

Design principle. *Keep identity and storage monotone and complete; spend parsimony in views.* Monotone storage buys Theorems 2–6 — stable references, semi-naive deltas, canonical checkpoints, streaming. Parsimony is real but representational; the trilemma says it cannot be made extensional in storage without forfeiting growth.

7. Worked growth run — the machine-checked ledger

Reference implementation: `growth_check.py` (pure Python 3 standard library: `hashlib`, `iter-tools`, `collections`). Reproduce with `python3 growth_check.py`. **All 15 checks pass.** Demonstration tower as in §2; code parameters $b = 2, o = 2, r = 0$, so $m_0 = 3$.

A — Primary worked field. Seed: three atoms on D_1 , one on D_2 , one on D_3 . Only the D_1 cohort binds (size 3); the closure carries it to Root and closes to $\top$. The D_2/D_3 atoms are inert orphans.

part	kind	rule	descr	addr (first 12 hex)
a	Atom	input	D_1	0e695abf2881
a	Atom	input	D_1	3c7f6d70e099
a	Atom	input	D_1	c7b946f36c7d
a	Atom	input	D_2	f6f6adf60b22
a	Atom	input	D_3	6554a14f87c5
B	Block	bind	D_1	900b1087dd65

part	kind	rule	descr	addr (first 12 hex)
S	Section	lift	D_1	17cd9dd60db5
R	Root	frame	D_1	bf358e13a3c4
$\top$	const	—	—	07b4ed8e4e4e

Fingerprint 1066bddea3ff771058f4af6afcf828efbeb29f2968cf3c68cce02110cf64d9662. Addresses injective (incl. $\top$). $\text{ecc}(\top) = 4$ on the reached component; 2 orphans $\Rightarrow$ **ill-formed** — the picture Proposition 8 repairs.

B — Ingestion confluence. Base $\{b_1, b_2\}$ on D_1 (no binding yet); delta $\{d_1, d_2, d_3\}$ on D_1 . All 13 ordered set partitions of the delta, replayed by frontier-local incremental closure, yield **1 distinct fingerprint**, equal to the single-batch fingerprint 5c8942ed83c9...e76d38a. The 5-atom cohort closes to $16 = \binom{5}{3} + \binom{5}{4} + \binom{5}{5}$ blocks — confluence and the clique semantics in one run.

C — Absorption. Ill-formed (1 orphan) $\rightarrow$ well-formed (0 orphans, $\text{ecc}(\top) = 4$) via a 2-atom delta completing the D_2 cohort; stage-1 address set $\subset$ stage-2 address set; incremental = batch fingerprint. Hashes as in §4.

D — Non-monotone counterexample. $\Phi_{\max}(S_3) = \{B_{123}\}$, $\Phi_{\max}(S_4) = \{B_{1234}\}$: B_{123} (addr 362db034e4f1...) **retracted**, monotonicity check False; complete closure on the same growth: B_{123} present at both stages, conservativity True.

E — Quotient view. Field blocks $1 \rightarrow 5$, view blocks $1 \rightarrow 1$ across the g_7 ingestion; view non-conservative True, field conservative True.

F — Clique counts. $n = 3, 4, 5, 6 \Rightarrow 1, 5, 16, 42$ blocks, each matching $\sum_{m \geq 3} \binom{n}{m}$; sections and roots track blocks one-for-one.

G — Betti number. $n = 6$ field: $V = 133$, $E = 282$, $C = 1$, $\beta_1 = 150$; union-find replay matches; $\Delta\beta_1 = m - j$ at every ascent instance.

H — Threshold grid. Over the 11×11 integer grid $(b, o) \in \{1..11\}^2$, $r = 0$: brute-force least admissible m equals $\lfloor o/b \rfloor + 2$ in every cell, and $m_0 = 3 \iff o/b \in [1, 2)$. Samples: $(o=2, b=2)$: $\Delta(2) = 0$, $\Delta(3) = 2$, $m_0 = 3$; $(o=4, b=2)$: $\Delta(3) = 0$, $\Delta(4) = 2$, $m_0 = 4$.

Ledger summary. A: injectivity $\checkmark$ · B: 13 histories $\rightarrow$ 1 fingerprint $\checkmark$, incremental = batch $\checkmark$ · C: absorption $\checkmark$, conservativity $\checkmark$, incremental = batch $\checkmark$ · D: $\Phi_{\max}$ non-monotone $\checkmark$, complete closure conservative $\checkmark$ · E: view non-conservative $\checkmark$, field conservative $\checkmark$ · F: $n=6 \rightarrow 42$ $\checkmark$ · G: replay = direct $\checkmark$, per-instance law $\checkmark$ · H: grid $\checkmark$, characterization $\checkmark$ — **15/15 PASS**.

8. Empirical scale and mechanized verification

All measurements in this section are produced by `bench_growth.py` and `corpus_cohorts.py` (pure stdlib + matplotlib) against the unmodified reference implementation; rerun both to reproduce every number and figure.

8.1 Delta integration cost — Theorem 4’s remark, measured

Random corpora with every descriptor cohort capped at 5 atoms (so the complete closure stays small per cohort); a fixed delta of $|D| = 20$ atoms, half landing in existing cohorts to exercise absorption; median of 5 runs. At every size the incremental and batch fingerprints are asserted equal before timing is reported.

$ S $	parts in $K(S \cup D)$	batch reclosure (s)	incremental (s)	speedup
250	3,546	0.035	0.006	5.9×
500	6,289	0.080	0.009	9.3×
1,000	11,496	0.154	0.015	10.2×
2,000	22,096	0.310	0.032	9.6×
4,000	43,296	0.606	0.067	7.5–9.6×
8,000	85,696	1.241	0.166	7.5×

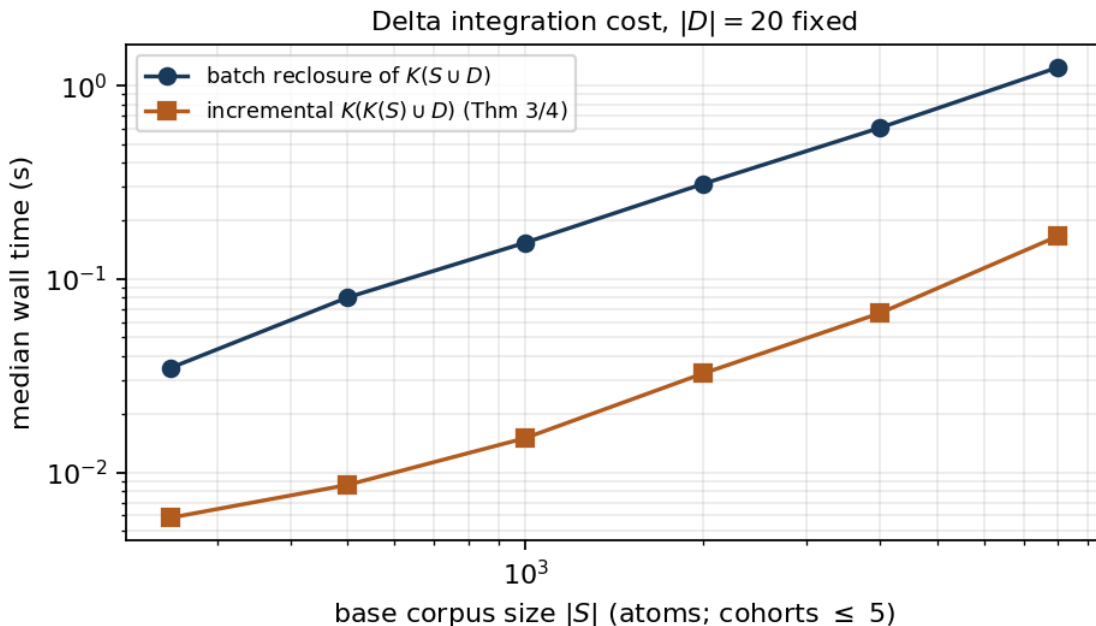


Figure 1: Median wall time for integrating a fixed 20-atom delta: batch reclosure grows linearly with the base; frontier-local incremental closure stays an order of magnitude below it.

Batch cost grows linearly with $|S|$, as it must — it re-derives the whole field. The incremental curve’s residual growth is an artifact of the deliberately unoptimized reference implementation, which rebuilds its atom index over all atoms once per frontier round; with premise indexing that term vanishes and the cost is proportional to the delta closure alone, which is exactly the cost statement accompanying Theorem 4. The honest headline is the measured one: a 6–10× advantage with no indexing at all, and bit-identical fingerprints throughout.

8.2 Descriptor cohorts on a natural corpus — does Proposition 10 bite?

A reproducible natural-language stress proxy: atoms are the 8,631 sentences of *Moby-Dick* (Project Gutenberg #2701, header/footer stripped, sentences of ≥ 20 characters); the descriptor of a sentence is its most frequent non-stopword token of length ≥ 3 , ties broken alphabetically. The map is deliberately crude — the point is the *shape* of naturally occurring cohorts, not linguistic quality.

The result: 3,078 descriptors; 637 bindable cohorts ($n \geq 3$) covering 5,671 atoms (65.7%); a heavy Zipf-like tail with maximal cohort $n = 268$ (the descriptor *ahab*). Under the complete

bind semantics the implied block count is $\sim 10^{80.7}$ — dominated by the maximal cohort alone, and beyond physical storage. The quotient view of §6.1 materializes 637 blocks; the cohort-object representation of §6.2 stores 3,078 records.

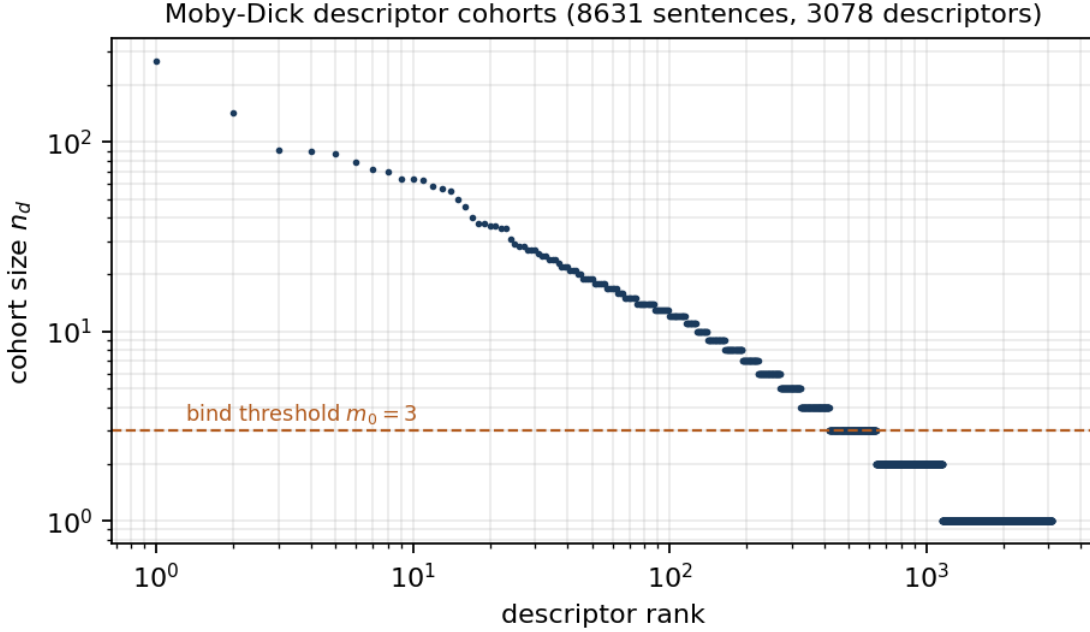


Figure 2: Cohort sizes by descriptor rank (log-log): a heavy tail with one 268-atom cohort. Everything above the dashed $m_0 = 3$ line is bindable.

The reading is sharp: under natural heavy-tailed descriptor maps, the complete semantics is not merely expensive but unrealizable in storage, so the §6 resolutions are forced, not optional. Crucially, *complete identity* survives: every one of the $\sim 10^{80.7}$ bindings has a well-defined address computable on demand (H4 is a function, not a table), so the field remains canonical while only a linear-size representation is ever materialized — the design principle of §6 made quantitative.

8.3 Verification status

Rung	Artifact	Establishes	Result
0	this paper	deductive proofs of all theorems	author-verified
1a	growth_check.py	15 fixed instance checks with real SHA-256 fingerprints	15/15 PASS
1b	test_growth_properties.py	14 universally quantified properties $\times$ 60 randomized adversarial instances each, with shrinking	14/14 PASS

Rung	Artifact	Establishes	Result
2	GrowthClosure.lean	kernel-checked proofs of Thms 1–6 (Thm 4 over the concrete RuleFamily format), Kleene stages, the Galois adjunction, the descent identity, Thm 11, and the H1/H2 discharge theorems; zero imports, zero sorry	compiles clean (Lean 4.30.0)

The axiom audit, verbatim from the compiler: conservativity, galois, descent, and the two discharge theorems `RuleFamily.step_mono` and `RuleFamily.step_finitary` depend on **no axioms** (pure constructive logic); `K_idem`, `incremental`, `ingest_eq`, `history_independent`, `continuity`, `stage-Union_eq_K`, `seminaive_eq`, and `seminaive_incremental` use only Lean’s standard kernel axioms `propext` and `Quot.sound` (via `funext`), solely because they are stated as set equalities; `Impossibility.impossibility` uses `propext` only. No `Classical.choice` appears anywhere — the development is constructive, including the Theorem 4 proof, whose only combinatorial ingredients are two finite-list lemmas (a per-element disjunction split and a uniform stage bound), neither needing excluded middle. The single assumption remaining outside the machine-checked perimeter, by design: the cryptographic layer of H4 (collision resistance).

9. Related work

Fixpoints and least models. Existence and constructivity are Knaster-Tarski (Tarski 1955) and Kleene; the closure $K(S)$ is the least-Herbrand-model construction of logic programming (van Emden & Kowalski 1976) transported to typed graphs.

Semi-naive evaluation and incremental view maintenance. Theorem 4 is the semi-naive principle of Datalog evaluation (Bancilhon 1986; Ullman 1989) restated for content-addressed parts. The contrast with deletion-capable maintenance (DRed; Gupta, Mumick & Subrahmanian 1993) is structural rather than algorithmic: conservativity (Thm. 2) removes deletions from the model, so the over-delete/rederive machinery is not optimized away but *absent*.

Monotonicity boundaries. The CALM conjecture (Hellerstein 2010), proved via relational transducer networks (Ameloot, Neven & Van den Bussche 2013) and surveyed in Hellerstein & Alvaro (2020), equates coordination-free consistency with logical monotonicity; weakened monotonicities enlarge the coordination-free class (Ameloot, Ketsman, Neven & Zinn 2015; Zinn, Green & Ludäscher 2012). Non-monotonicity of aggregates in recursion is classical (Ross & Sagiv 1992/1997). Theorem 11 instantiates this boundary in the content-addressed MDL format; the positioning paragraph of §5 gives the full dictionary.

The chase. The binding rules are tuple-generating dependencies firing on every satisfying homomorphism; the closure is a chase (Fagin, Kolaitis, Miller & Popa 2003/2005). Content addressing acts as collision-resistant Skolemization — the generated null’s name is a function of the generating tuple — so H4 places the system in the Skolem/semi-oblivious chase regime (Marnette 2009), where confluence (Lemma D, Thm. 5) is exactly chase determinism for functional Skolem terms.

MDL. The admissibility law is two-part-code minimum description length (Rissanen 1978; Grünwald 2007). Corollary 12 formalizes a known informal tension: context-sensitive codebooks are globally optimal but non-monotone; payload-determined codes are locally greedy but compositional.

Content-addressed storage and replication. H4 and Definition 6 are the Merkle-DAG discipline (Merkle 1987) underlying Git and IPFS; Theorem 2(iii) is the Merkle diff. Theorems 2–6 jointly state that the field is a grow-only replicated set (G-Set CRDT) over derivation records — union-merge, hence convergent — which is why ingestion order can never matter.

Merkle-replicated state. Merkle-CRDTs (Sanjuán, Pöyhtäri, Teixeira & Psaras 2020) combine Merkle-DAGs with join-semilattice merge for convergent replication; the present work differs in studying the *generation* of the DAG by a rule closure, its growth theory, and the parsimony impossibility, not replication.

Selection as set cover. Phase 2 of §6.3 is weighted set cover with the classical greedy guarantee (Chvátal 1979).

10. Limitations

1. **H2 excludes context-sensitive MDL.** Payload-determined codes cannot price a part against the *global* codebook; Theorem 11 shows this is the exact boundary of the monotone regime, not an artifact of the proof technique. Globally optimal compression belongs in Phase-2 views (§6.3).
 2. **Finiteness requires bounded grade.** $K(S)$ is finite for finite S only because ascent is strictly grade-increasing over a finite tower and admissibility gates the lateral/binding combinatorics; unbounded towers reintroduce infinite closures.
 3. **Complete semantics is exponential in cohort size.** Proposition 10 is a theorem about the chosen semantics, and the resolutions of §6 are mitigations of representation, not of the count itself.
 4. **Status.** Working paper; the proofs are elementary but have not been independently refereed. The implementation checks the quantitative claims on the demonstration tower; the theorems themselves are stated for any system satisfying H1–H4.
-

11. Conclusion

A content-addressed rule closure can be grown the way a ledger is grown: append-only, order-blind, and auditable. Four hypotheses — finitary rules, payload-determined admissibility, stratified acyclic generation, content addressing — buy four guarantees: nothing is ever retracted and growth is a Merkle diff; deltas close incrementally from the frontier with no rederivation; the checkpoint fingerprint is a function of content, never of history; and streaming is exact. The dynamics those guarantees induce are themselves lawful: admissibility only switches on, repair is absorption by seed growth, well-formedness is a cheap re-checkable predicate rather than an invariant, and the topology only accumulates cycles. The one thing such a system cannot do is *also* store exactly the maximal, non-redundant structure: maximality is a property of the ambient seed, provably inexpressible in payload-determined side conditions, and insisting on it in storage forfeits monotonicity, stable identity, and history independence at once. The trilemma resolves by separation of concerns — identity and storage monotone and complete; parsimony in views — and every claim in this account is carried by a three-rung ladder: a 666-line standard-library ledger (15/15 PASS), a randomized property suite (14/14 on ~840 instances), and a kernel-checked, constructive Lean 4 development in

which the closure algebra and the impossibility witness are theorems — with the boundary itself now positioned where it belongs, as the content-addressed face of CALM.

References

Modern entries were existence-verified (venue, year, pages where shown) during the v2 prior-art sweep; classical entries are standard.

1. A. Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific J. Math.* 5(2):285–309, 1955.
2. S. C. Kleene. *Introduction to Metamathematics*. North-Holland, 1952.
3. M. H. van Emden, R. A. Kowalski. The semantics of predicate logic as a programming language. *J. ACM* 23(4):733–742, 1976.
4. F. Bancilhon, R. Ramakrishnan. An amateur’s introduction to recursive query processing strategies. *SIGMOD* 1986.
5. J. D. Ullman. *Principles of Database and Knowledge-Base Systems*, Vol. II. Computer Science Press, 1989.
6. A. Gupta, I. S. Mumick, V. S. Subrahmanian. Maintaining views incrementally. *SIGMOD* 1993, 157–166.
7. R. Fagin, P. G. Kolaitis, R. J. Miller, L. Popa. Data exchange: semantics and query answering. *ICDT 2003*, 207–224; *Theor. Comput. Sci.* 336(1):89–124, 2005.
8. B. Marnette. Generalized schema-mappings: from termination to tractability. *PODS* 2009, 13–22.
9. J. Rissanen. Modeling by shortest data description. *Automatica* 14(5):465–471, 1978.
10. P. D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.
11. R. C. Merkle. A digital signature based on a conventional encryption function. *CRYPTO* 1987.
12. V. Chvátal. A greedy heuristic for the set-covering problem. *Math. Oper. Res.* 4(3):233–235, 1979.
13. M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. Conflict-free replicated data types. *SSS* 2011.
14. J. M. Hellerstein. The declarative imperative: experiences and conjectures in distributed logic. *SIGMOD Record* 39(1), 2010 (PODS 2010 keynote).
15. T. J. Ameloot, F. Neven, J. Van den Bussche. Relational transducers for declarative networking. *PODS* 2011, 283–292; *J. ACM* 60(2), Art. 15, 2013.
16. J. M. Hellerstein, P. Alvaro. Keeping CALM: when distributed consistency is easy. *Commun. ACM* 63(9), 2020.
17. T. J. Ameloot, B. Ketsman, F. Neven, D. Zinn. Weaker forms of monotonicity for declarative networking: a more fine-grained answer to the CALM-conjecture. *PODS* 2014, 64–75; *ACM TODS* 40(4), Art. 21, 2015.
18. D. Zinn, T. J. Green, B. Ludäscher. Win-move is coordination-free (sometimes). *ICDT* 2012.
19. K. A. Ross, Y. Sagiv. Monotonic aggregation in deductive databases. *PODS* 1992, 114–126; *J. Comput. Syst. Sci.* 54(1):79–97, 1997.
20. H. Sanjuán, S. Pöyhtäri, P. Teixeira, Y. Psaras. Merkle-CRDTs: Merkle-DAGs meet CRDTs. arXiv:2004.00107, 2020.

Revision note — v1 → v2 → v2.1

1. **Theorem 11 repositioned.** Following a prior-art sweep, the impossibility result is now stated as a content-addressed, MDL-flavored instance of the CALM boundary (positioning paragraph, §5; dictionary in §9), with the trilemma formulation, the MDL localization, and the mechanized witness as the claimed contributions. Abstract and contributions adjusted accordingly.

2. **Verification ladder added** (§8.3): randomized property suite (14/14, ~840 instances) and a self-contained, constructive, sorry-free Lean 4 formalization with verbatim axiom audit. v1 predated both.
3. **Empirical scale added** (§8.1-8.2): measured 6-10× incremental advantage on bases up to 8,000 atoms with fingerprint-equality assertions, and the Moby-Dick cohort experiment showing the complete semantics implies $\sim 10^{80.7}$ bindings on natural heavy-tailed descriptors — making §6 quantitatively forced.
4. **Bibliography formalized.** Informal author-year mentions replaced by a 20-entry reference list; eleven modern entries existence-verified during the sweep.
5. **Related work extended** with the CALM cluster, weakened monotonicity, Ross-Sagiv aggregation, Marnette’s chase analysis, and Merkle-CRDTs.
6. **v2.1 — machine-checked perimeter completed.** Theorem 4 is kernel-checked over a concrete payload-determined rule format (`RuleFamily`): cumulative frontier stages (`fstage`) that never fire an all-old instance provably reach the full incremental closure (`seminaive_eq`, `seminaive_incremental`), via a constructive Kleene-stage development (`stageUnion_eq_K`). $H1 \Rightarrow$ finitariness and $H2 \Rightarrow$ monotonicity are now theorems, both **axiom-free** (`RuleFamily.step_finitary`, `RuleFamily.step_mono`), discharging the standing hypotheses of v1-v2. The only assumption left outside the checked perimeter is SHA-256 collision resistance, by design.

Artifacts: `growth_check.py` (ledger) · `test_growth_properties.py` (property suite) · `GrowthClosure.lean` (kernel-checked proofs) · `bench_growth.py`, `corpus_cohorts.py` (§8 measurements) · all deterministic and reproducible from the repository.